

CURATING A ROBUST LLM POOL: A TWO-STAGE STOCHASTIC OPTIMIZATION APPROACH FOR COST-CONSTRAINED LLM ROUTING

Jingwen Yang, Yu Hin Liang
INDENG 164, Spring 2026

ABSTRACT

LLM routing asks which model to call on each user prompt. We cast this problem for an agentic AI company as a two-stage stochastic mixed-integer program. The first stage selects a small *pool* of models to maintain in production, and the second stage routes each realised prompt to one of the selected models. Sample-average approximation on `routerbench` (240 prompts, 32 models, four benchmark domains) produces a tractable MILP. We add four realistic deployment constraints (a pool-size cap, a storage budget, a per-prompt fairness slack, and provider rules) and a distributionally robust extension that hedges against shifts in the prompt mixture. The resulting policy traces a smooth Pareto frontier on the cost-quality plane. At a budget of \$0.001 per prompt the policy reaches a weighted score of 0.967, which already exceeds the best single model GPT-5 by eight percentage points at fifty times lower cost. Saturating the budget at \$0.005 per prompt drives the score to the per-prompt oracle of 0.983, a thirteen-fold cost reduction relative to running GPT-5 on every prompt. A pool of $K = 6$ already captures 99.7% of the achievable quality, and a $\pm 70\%$ shift in benchmark proportions costs at most 1.2 percentage points of worst-case quality. Code is available at <https://github.com/Bi8bo001/indeng164-project>.

1 INTRODUCTION

We adopt the role of an operations-research scientist at *CodePilot AI*, an agentic AI company that ships a Cursor-style coding assistant and a tutoring sidebar for STEM students. Its workload spans code, mathematics, and knowledge questions. For every prompt the company picks one of about thirty candidate LLMs, ranging from GPT-5 at \$0.04 per prompt down to free open-source 9 B-parameter backbones. Choosing well, prompt by prompt, is the difference between profit and loss.

Two decisions live on different time scales. *Pool curation* selects which models to maintain in production and binds for weeks because each model carries integration, monitoring, and storage cost. *Routing* selects which model to call on a given prompt and repeats over a stream of stochastic future requests. The best pool depends on the routing policy that will operate on it, and the best policy in turn depends on which models the company actually maintains. This coupling is the structure of a two-stage stochastic program with recourse (Birge and Louveaux, 1997): the pool y is first-stage and is committed before the prompt distribution is realised, while the routing x_ξ is second-stage and is made after each prompt is observed. The prompt distribution $\mathbb{P}$ itself is unknown, and we observe only $|\mathcal{P}| = 240$ samples from it, so we approximate expectations through sample-average approximation.

Two questions then drive the report. They are precisely the questions a deployment manager would ask before signing off on a routing system: **RQ1: pool curation.** How many models, and which ones, should the company maintain under a fixed inference budget, and are there diminishing returns as the pool grows? **RQ2: distributional robustness.** If the prompt mix shifts in production, how much worst-case quality is lost, and is the optimal pool stable across such shifts?

2 DATA AND NOTATION

The `routerbench` dataset (Li et al., 2026) provides binary correctness $Q_{pm} \in \{0, 1\}$ and per-prompt cost $C_{pm} \in [0, \$0.108]$ for 33 models on 240 prompts. The prompts are equally split across four benchmarks (AIME, LCB, GPQA, MMLU-PRO). One model (DEEPSEEK-V3.1-TERMINUS (Liu et al., 2024)) covers only 180 of the 240 prompts; we discard it and work with the $|\mathcal{M}| = 32$ fully-covered models to avoid ad hoc imputation. Nine of the remaining models have $C_m = 0$ because they are open-source and can be served locally. The most expensive model is GPT-5, at \$0.0425 per prompt and weighted score 0.888 on the dataset. We write $\mathcal{P}_d$ for the prompt subset of domain $d \in \mathcal{D}$, w_p for the SAA prompt weight (uniform $1/|\mathcal{P}|$ at baseline), S_m for the storage footprint of model m in GB (zero for API-only models), and use $(B, K, S^{\max}, \tau, \lambda)$ for the cost budget, pool cap, storage cap, fairness target, and slack price.

3 OPTIMIZATION MODEL

Decision variables. The first-stage variables $y_m \in \{0, 1\}$, $m \in \mathcal{M}$, encode whether model m is selected into the production pool. The second-stage variables $x_{pm} \in \{0, 1\}$ encode the routing decision for each prompt. A non-negative slack $s_p \geq 0$ softens the per-prompt fairness target in cases where the data does not allow it to be met exactly.

Population two-stage program. For a random prompt $\xi \sim \mathbb{P}$, the routing policy is a function $x(\xi)$ from prompts to feasible assignment vectors. The population optimisation is

$$\begin{aligned} \max_{y, x(\cdot)} \mathbb{E}_\xi [\sum_m Q(\xi, m) x_m(\xi)] \quad \text{s.t.} \quad & \sum_m x_m(\xi) = 1, \quad x_m(\xi) \leq y_m \quad \text{a.s.}, \\ & \sum_m y_m \leq K, \quad \sum_m S_m y_m \leq S^{\max}, \quad \mathbb{E}_\xi [\sum_m C(\xi, m) x_m(\xi)] \leq B. \end{aligned} \quad (1)$$

The budget is imposed in expectation rather than per-realisation to match a monthly inference budget; y is first-stage because it is committed before any prompt is seen, and x is second-stage because it depends on ξ .

Sample-average approximation (BC-2SP). The true distribution $\mathbb{P}$ is unknown, so we apply sample-average approximation (Shapiro et al., 2021). We replace $\mathbb{P}$ by the empirical distribution on the 240 observed prompts with weights w_p and write $x_{pm} \equiv x_m(\xi = p)$. The deterministic-equivalent MILP is

$$\begin{aligned} \max_{x, y, s} \quad & \sum_{p, m} w_p Q_{pm} x_{pm} - \lambda \sum_p w_p s_p - \varepsilon \sum_{p, m} w_p C_{pm} x_{pm} \\ \text{s.t.} \quad & \sum_m x_{pm} = 1, \quad x_{pm} \leq y_m, \quad \sum_m y_m \leq K, \quad \sum_m S_m y_m \leq S^{\max}, \\ & \sum_{p, m} w_p C_{pm} x_{pm} \leq B, \quad \sum_m Q_{pm} x_{pm} + s_p \geq \tau, \quad x_{pm}, y_m \in \{0, 1\}, \quad s_p \geq 0. \end{aligned} \quad (2)$$

The first term is the SAA estimator of expected quality. The second term softly penalises violation of the per-prompt fairness target τ , with slack price λ chosen so that the routing prefers a correct answer whenever one exists. The third term, with $\varepsilon = 10^{-3} \ll \lambda$, is a cost tie-breaker that selects the cheapest among quality-equivalent solutions and removes a non-monotonicity along the upper frontier. Provider rules are encoded by fixing the corresponding y_m .

Distributionally robust extension (DRO BC-2SP). The empirical proportions $w_d^0 = |\mathcal{P}_d|/|\mathcal{P}|$ are themselves a noisy estimate of the production benchmark mixture $w_d := \Pr[\xi \in \mathcal{P}_d]$, and a router that is tuned to w^0 may degrade under a future shift. Motivated by distributionally robust optimisation more generally (Mohajerin Esfahani and Kuhn, 2018), we hedge against this by modelling the true mixture as lying in a simple *relative-box* ambiguity set (rather than a Wasserstein ball) $\mathcal{U}(\delta) = \{w \in \Delta_{|\mathcal{D}|} : |w_d - w_d^0| \leq \delta w_d^0 \forall d \in \mathcal{D}\}$ around w^0 , while keeping prompts within each domain uniformly weighted. Writing $f_d(x) := \frac{1}{|\mathcal{P}_d|} \sum_{p \in \mathcal{P}_d, m} Q_{pm} x_{pm}$ for the conditional expected score on domain d , the DRO objective becomes $\max_{y, x} \min_{w \in \mathcal{U}(\delta)} \sum_d w_d f_d(x)$. The inner minimisation over w is a small LP that admits a closed form. Reparametrising $w_d = w_d^0(1 + e_d)$ with $|e_d| \leq \delta$ and $\sum_d w_d^0 e_d = 0$ and applying LP duality gives

$$\min_{w \in \mathcal{U}(\delta)} \sum_d w_d f_d(x) = \sum_d w_d^0 f_d(x) - \delta \cdot \min_{\lambda \in \mathbb{R}} \sum_d w_d^0 |f_d(x) - \lambda|, \quad (3)$$

where the inner minimiser λ^* is the w^0 -weighted median of $\{f_d(x)\}$. Linearising the absolute value with non-negative auxiliaries z_d and a free λ turns the outer max into the MILP

$$\max_{x, y, \lambda, z} \sum_d w_d^0 f_d(x) - \delta \sum_d w_d^0 z_d \quad \text{s.t.} \quad z_d \geq f_d(x) - \lambda, \quad z_d \geq \lambda - f_d(x), \quad \forall d \in \mathcal{D}, \quad (4)$$

together with the BC-2SP constraints in (2) and the budget imposed under the nominal w . The reformulation adds only $|\mathcal{D}| + 1$ continuous variables and $2|\mathcal{D}|$ inequality constraints, so DRO BC-2SP is essentially the same size as BC-2SP.

Companion formulations. We also consider three variants that share the constraint set of BC-2SP and stress different aspects of the trade-off. *QC-2SP* minimises expected cost subject to a target weighted score $Q^{\min}$, and traces the same Pareto frontier from the dual side. *Randomised BC-2SP* relaxes $x_{pm} \in \{0, 1\}$, so that the recourse becomes an explicit stochastic policy that selects model m on prompt p with probability x_{pm} . *Cascade BC-2SP* pairs each prompt with a tuple (m_1, m_2) and a zero-cost verifier: model m_1 is invoked first, and the system escalates to m_2 only on failure, which gives a MILP with $|\mathcal{P}||\mathcal{M}|^2$ binary variables.

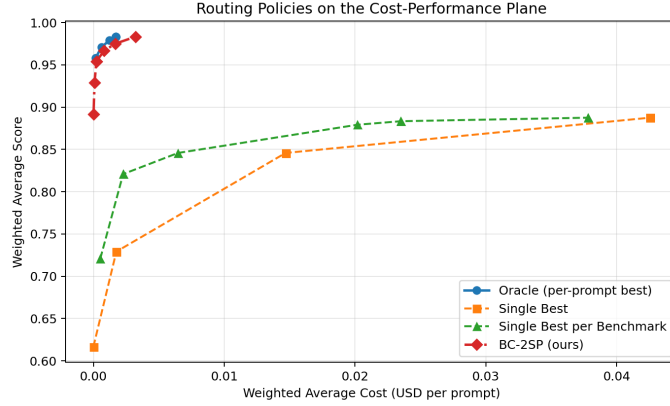


Figure 1: Cost-quality Pareto frontier on `routerbench`. Oracle is the per-prompt unconstrained optimum and serves as an upper bound on any pool-constrained policy. BC-2SP (red) reaches the oracle score with a pool of $K = 8$ and dominates both Single-Best baselines by an order of magnitude in cost.

Solving the MILP. The BC-2SP MILP has 8,224 binary variables and 8,163 linear constraints. We implement BC-2SP, QC-2SP, DRO BC-2SP and the cascade variant in Pyomo (Bynum et al., 2021) with the HiGHS solver (Huangfu and Hall, 2018); a single instance solves in two to four seconds, and a full eleven-point frontier sweep completes in under one minute. Storage parameters S_m are heuristics from the model name: roughly 14 to 18 GB for 7 to 9 B open-source backbones, 0 GB for provider-hosted APIs, and about 470 GB for the 235 B Qwen mixture-of-experts. The notebook `solution.ipynb` accompanies this report.

4 EXPERIMENTS

All experiments use uniform empirical weights $w_p = 1/240$, $\varepsilon = 10^{-3}$, $\lambda = 1$, and $\tau = 1$ unless otherwise stated; under $\tau = 1$ the slack s_p records whether prompt p is left unsolved by its assigned model.

Cost-quality Pareto frontier. We initially adopted the weighted-sum scalarisation $\min \text{cost} - \alpha \cdot \text{score}$ suggested by the starter code, but found that α has no operational meaning and that combining it with a fairness penalty collapses the pool to two or three models at small α . Switching to an explicit budget B removes both issues: the budget has clear monetary meaning, and sweeping it from $\$10^{-5}$ to $\$5 \times 10^{-3}$ at $K = 8$ traces the BC-2SP frontier of Figure 1, which strictly dominates every Single-Best baseline. At $B = \$10^{-5}$ the policy already scores 0.892, marginally above GPT-5 but at four orders of magnitude lower cost, because the optimal pool concentrates on free open-source backbones. At $B = \$10^{-3}$ the score reaches 0.967, which is eight percentage points above GPT-5 at fifty times lower cost. At $B = \$5 \times 10^{-3}$ the policy attains the per-prompt oracle quality of 0.983 at actual cost \$0.0032 per prompt, a 13.1 $\times$ cost reduction relative to GPT-5. Beyond this budget the score saturates because no further quality is available in the data. Table 1 summarises the headline operating points.

Policy	Score	Cost (\$/prompt)	Pool size $ y $
Single Best (GPT-5)	0.8875	0.04255	1
BC-2SP, $B = \$10^{-4}$	0.9292	0.000089	8
BC-2SP, $B = \$10^{-3}$	0.9667	0.000822	8
BC-2SP, $B = \\$5 \times 10^{-3}$	0.9833	0.00323	8
Oracle (per-prompt best)	0.9833	0.00170	32

Table 1: Headline operating points along the BC-2SP frontier, against GPT-5 (best single model) and the per-prompt oracle (unconstrained upper bound). The bolded row is our recommended setting.

RQ1: diminishing returns of pool size. Fixing $B = \$5 \times 10^{-3}$ and sweeping K from 1 to 10 (Figure 2a) shows a fast-then-saturating curve. Under this budget a single-model pool achieves only 0.742, because the strongest single model GPT-5 is not budget-feasible as an always-on choice (it costs \$0.0425 per prompt) and no budget-feasible single model covers all four benchmarks adequately. Adding a second model already lifts the score to 0.892 (a fifteen percentage-point gain). The curve continues to rise but flattens quickly: $K = 4$ reaches 0.946, $K = 6$ reaches 0.979 (99.6% of the $K = 10$ value), and from $K = 7$ on the score is pinned at the oracle 0.983. The operationally relevant message is that six to seven well-chosen models are sufficient regardless of how many candidates the company nominally has access to; further models are pure overhead.

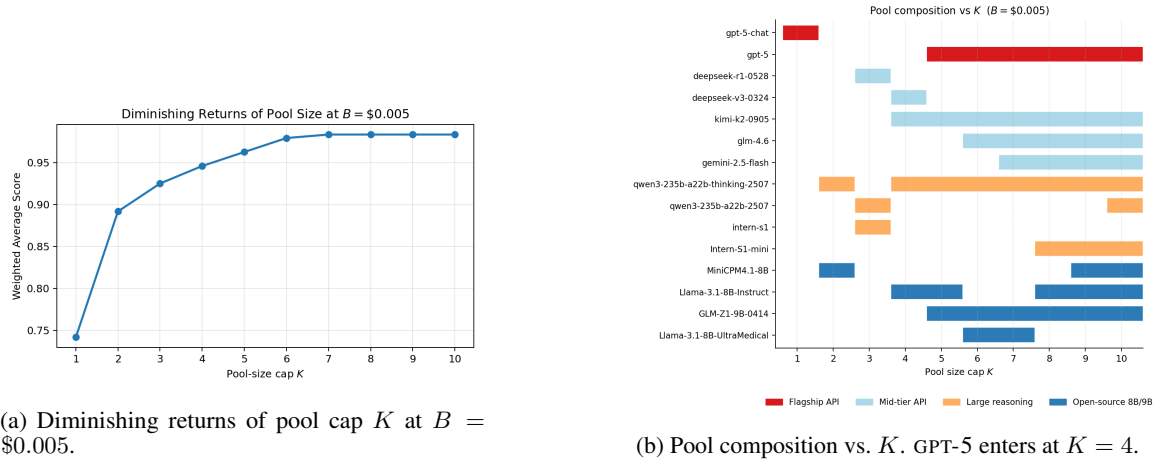


Figure 2: Quality saturates at $K = 7$, and a $K = 6$ pool already captures 99.7% of the achievable quality.

Pool composition and traffic concentration. The growth path of the optimal pool, shown in Figure 2b, is itself informative. The optimiser first selects complementary free open-source backbones; only at $K = 4$ does it admit a flagship; and at larger K it fills in with mid-tier APIs. The composition is therefore not “small models on top of one big model” but the reverse: a mostly free base, with a flagship pulled in only when the data demands it.

The traffic distribution at $K = 8$ tells the same story from a different angle. The single most-used model is the free GLM-Z1-9B-0414 (GLM Team et al., 2024), which alone handles 57.5% of prompts. The remaining traffic spreads across six models, with no single model exceeding 11%: KIMI-K2-0905 (10.0%), INTERN-S1-MINI (8.3%), LLAMA-3.1-8B-INSTRUCT (Grattafiori et al., 2024) (7.1%), GLM-4.6 (5.8%), GEMINI-2.5-FLASH (5.4%), and QWEN3-235B-THINKING-2507 (Yang et al., 2025) (4.2%). GPT-5 sits at the bottom of this list at 1.7%, used only as a backstop on the four hardest prompts that no smaller model can solve. The optimal routing is therefore flat-with-a-backstop rather than concentrated on the strongest single model.

Per-benchmark breakdown. At $B = \$0.005$, $K = 8$ the policy decomposes as in Table 2: LCB and GPQA are solved perfectly, while AIME and MMLU-PRO both reach 0.967. The four remaining prompts are exactly those for which no model in the data produces a correct answer, so the routing matches the per-prompt oracle quality on every solvable prompt.

Benchmark	AIME	LCB	GPQA	MMLU-PRO
Score	0.967	1.000	1.000	0.967
Cost (\$/prompt)	0.00592	0.00479	0.00136	0.00086

Table 2: Per-benchmark score and cost at $B = \$0.005$, $K = 8$ (60 prompts each).

RQ2: distributional robustness. Sweeping δ from 0 to 0.7 at $B = \$0.005$, $K = 8$ (Figure 3), the nominal score remains pinned at 0.983. The worst-case score from (4) drops by 0.50, 0.83, and 1.17 percentage points at $\delta = 0.3, 0.5, 0.7$. The DRO pool overlaps with the nominal in seven of eight slots for $\delta \leq 0.5$, so the pool itself is highly stable. The price of distributional robustness on this dataset is therefore small.

Constraint ablations and formulation comparison. The realistic constraints behave as one would hope. Storage caps down to 50 GB are free, while tightening to 20 GB costs 1.2 pp because the optimiser falls back from ~ 8 B-parameter open-source backbones onto smaller alternatives. Excluding the OpenAI flagship pair costs only 0.4 pp and actually *lowers* cost to \$0.0024 per prompt, because QWEN3-235B-THINKING-2507 and KIMI-K2-0905 absorb most of the lost quality; mandating CLAUDE-SONNET-4 preserves the score but raises cost by 38%. Across formulations, BC-2SP and QC-2SP coincide at the corresponding $(B, Q^{\min})$ pair as expected for a primal-dual pair on the same Pareto frontier, the DRO variant pays a small cost premium for δ -protection, and the randomised and cascade variants give only marginal lift on this binary-score dataset (randomised +0.35 pp at $B = \$0.001$; cascade matches BC-2SP at $B = \$0.0005$). Finer-grained graded scores would amplify the latter two in production.

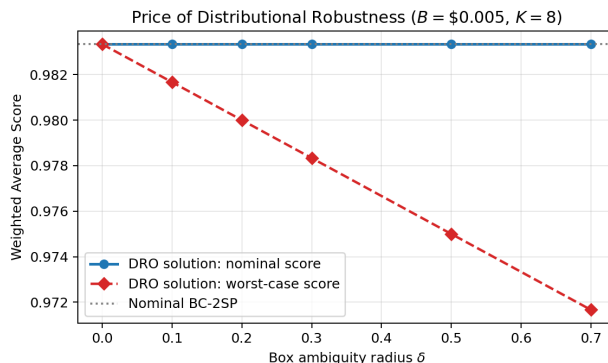


Figure 3: Price of distributional robustness. The worst-case score drops by about 1.2 percentage points at $\delta = 0.7$, while the nominal score stays at the oracle value.

5 DISCUSSION AND RECOMMENDATIONS

Answer to RQ1. A pool of $K^* = 6$ models is sufficient: it captures 99.7% of the quality of any larger pool we tested, and adding a seventh model already pushes the score onto the per-prompt oracle quality. Crucially, the pool that the optimiser produces is heterogeneous and not concentrated. GPT-5 routes only 1.7% of traffic, while free open-source backbones together handle about 60%. The cost-quality optimum on this benchmark is therefore obtained by routing most prompts through free models and reserving a single flagship as a backstop for the small set of hard prompts that no smaller model can solve.

Answer to RQ2. The cost of robustifying against benchmark-mix shifts is also small. Even when each benchmark proportion is allowed to swing by $\pm 70\%$, the worst-case score of the DRO solution drops by at most 1.2 pp; and for $\delta \leq 0.5$ the DRO pool overlaps with the nominal solution in seven of eight slots. Because the DRO MILP is essentially the same size as BC-2SP, deploying the DRO solution by default is essentially free insurance against mix shifts.

Recommendations. We recommend $K = 6$ as the cost-conscious production default since it already captures 99.7% of the achievable quality and saves the overhead of an extra slot; the figures and tables use $K = 8$ only as a diagnostic upper-frontier setting that matches the oracle quality exactly. A concrete instance of the six-model pool combines a flagship backstop, a mid-tier API, a reasoning model, and two or three free ~ 8 B open-source backbones, e.g. {GPT-5, GEMINI-2.5-FLASH, KIMI-K2-0905, GLM-Z1-9B-0414, INTERN-S1-MINI, LLAMA-3.1-8B-INSTRUCT}. The SAA MILP solves in seconds, so the team should re-solve it whenever the cost or quality target changes rather than freezing the policy; fairness should be enforced softly through s_p so that unsolvable prompts are absorbed by the slack; and a storage budget of 50 GB is sufficient (tighter caps cost about 1.2 pp).

Limitations and future work. We follow the dataset convention and treat the nine open-source models as having zero per-prompt API cost; in practice, GPU serving carries amortised hardware, latency, and maintenance cost that could be added as a separate cost term, on top of the storage cap we already model. Beyond that, binary correctness underuses information that graded scores would expose, which is precisely the regime where the randomised and cascade variants should outperform deterministic routing. Latency and log-loss objectives drop into the SAA MILP without difficulty, and an online-learning loop that updates $\hat{\mathbb{P}}$ from production traffic would let the router drift with the user base rather than freezing on historical samples.

Personal takeaway. Before this project we only had the rough intuition that small models should handle easy prompts and stronger ones be reserved for hard prompts. Writing out the two-stage formulation made this concrete: pool selection and per-prompt routing are two distinct decisions, and separating them showed us where the cost savings actually come from. The most surprising result was that a six-model pool nearly matches the per-prompt oracle quality at a small fraction of GPT-5’s cost, and HiGHS solving each instance in seconds made the work genuinely fun.

REFERENCES

- John R. Birge and François Louveaux. *Introduction to Stochastic Programming*. Springer, New York, NY, 1997.
- Michael L. Bynum, Gabriel A. Hackebeil, William E. Hart, Carl D. Laird, Bethany L. Nicholson, John D. Siirola, Jean-Paul Watson, and David L. Woodruff. *Pyomo — Optimization Modeling in Python*, volume 67. Springer, third edition, 2021.
- GLM Team, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, et al. ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools. *arXiv preprint arXiv:2406.12793*, 2024.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.
- Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, et al. LLMRouterBench: A massive benchmark and unified framework for LLM routing. *arXiv preprint arXiv:2601.07206*, 2026.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Peyman Mohajerin Esfahani and Daniel Kuhn. Data-driven distributionally robust optimization using the Wasserstein metric: Performance guarantees and tractable reformulations. *Mathematical Programming*, 171(1):115–166, 2018.
- Alexander Shapiro, Darinka Dentcheva, and Andrzej Ruszczyński. *Lectures on Stochastic Programming: Modeling and Theory*. Society for Industrial and Applied Mathematics, 2021.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

A DETAILED NUMERICAL RESULTS

Formulation	Setting	Score (cost \$/prompt)	Pool	Note
BC-2SP	$B = \$0.005$	0.9833 (0.00323)	8	main
QC-2SP	$Q^{\min} = 0.98$	0.9833 (0.00323)	8	dual matches BC-2SP
QC-2SP	$Q^{\min} = 0.95$	0.9500 (0.00019)	8	cheap operating point
QC-2SP	$Q^{\min} = 0.90$	0.9000 (0.0000179)	8	near-free quality
DRO BC-2SP, $\delta = 0.3$	$B = \$0.005$	0.9833 wc 0.978 (0.00500)	8	robust
DRO BC-2SP, $\delta = 0.7$	$B = \$0.005$	0.9833 wc 0.972 (0.00500)	8	wide ambiguity
Randomised BC-2SP	$B = \$0.001$	0.9702 (0.00100)	8	+0.35 pp vs det
Cascade BC-2SP	$B = \$0.0005$	0.9542 (0.00027)	8	matches BC-2SP
Single Best (GPT-5)	(−)	0.8875 (0.04255)	1	best single model
Single Best per Benchmark	(−)	0.8875 (0.03782)	4	one model per domain
Oracle (per-prompt best)	(−)	0.9833 (0.00170)	32	unconstrained upper bound

Table 3: Comparison of formulations and baselines at representative operating points. Cost in USD per prompt. “wc” denotes the worst-case score under the corresponding ambiguity radius.

Setting	Score	Cost (\$/prompt)	Pool	Δ vs baseline
BC-2SP free	0.9833	0.00323	8	(−)
no OpenAI (exclude GPT-5, GPT-5-CHAT)	0.9792	0.00240	8	−0.4 pp
must include CLAUDE-SONNET-4	0.9833	0.00444	8	+38% cost
$S^{\max} = 200$ GB	0.9833	0.00495	8	(−)
$S^{\max} = 100$ GB	0.9833	0.00485	8	(−)
$S^{\max} = 50$ GB	0.9833	0.00485	8	(−)
$S^{\max} = 20$ GB	0.9708	0.00441	8	−1.2 pp

Table 4: Constraint ablations at $B = \$0.005$, $K = 8$.

δ	nominal score	worst-case score	Δ vs nominal	λ^*
0.0	0.9833	0.9833	0.00 pp	0.000
0.1	0.9833	0.9817	−0.16 pp	1.000
0.2	0.9833	0.9800	−0.33 pp	1.000
0.3	0.9833	0.9783	−0.50 pp	1.000
0.5	0.9833	0.9750	−0.83 pp	1.000
0.7	0.9833	0.9717	−1.17 pp	0.967

Table 5: DRO BC-2SP sweep at $B = \$0.005$, $K = 8$. Worst-case score is the value of (4), and λ^* is the optimal weighted median that linearises the absolute values.